

Proyecto II – Programación II (EIF204)

Motor de Simulación de Aventuras y Mazmorras

Space Station Adventure Simulator

Estudiante encargado

- Johan Jafeth Zuñiga Rodriguez
-

1. Descripción General del Sistema

El sistema desarrollado consiste en un motor de simulación de aventuras en consola implementado en C++. La simulación representa la exploración de una estación espacial abandonada donde un personaje principal debe recorrer diferentes áreas, recolectar recursos, enfrentar eventos y sobrevivir hasta alcanzar el objetivo final.

La aplicación carga la información inicial desde archivos de texto, construye una representación interna del mundo y ejecuta una simulación dinámica donde las decisiones y eventos modifican constantemente el estado del personaje y del entorno.

2. Temática de la Aventura

La aventura se desarrolla en una estación espacial afectada por fallos estructurales y eventos peligrosos.

El protagonista, Commander Kael, debe recorrer distintas secciones de la estación buscando recursos y evitando amenazas para completar su misión.

Durante el recorrido pueden ocurrir diferentes situaciones:

- Encontrar objetos útiles.
 - Sufrir daños por eventos peligrosos.
 - Recuperar recursos.
 - Explorar nuevas áreas.
 - Alcanzar el objetivo final o perder la misión.
-

3. Archivos de Entrada

La simulación utiliza archivos de texto para construir el estado inicial del mundo.

rooms.txt

Contiene la información de las habitaciones de la estación.

Ejemplo:

Room1,ControlRoom Room2,EngineRoom

items.txt

Contiene los objetos disponibles dentro del mundo.

Ejemplo:

MedKit,20 OxygenTank,30

events.txt

Contiene los eventos que pueden ocurrir durante la aventura.

Ejemplo:

RadiationStorm,-15 AlienAttack,-20

4. Principales Clases del Sistema

World

Responsable de administrar el conjunto completo de habitaciones y conexiones del mundo.

Responsabilidades

- Almacenar habitaciones.
 - Gestionar conexiones.
 - Permitir navegación.
-

Room

Representa un espacio dentro de la estación espacial.

Responsabilidades

- Mantener objetos disponibles.
 - Mantener eventos asociados.
 - Gestionar conexiones con otras habitaciones.
-

Character

Representa al protagonista de la aventura.

Responsabilidades

- Administrar salud.
 - Administrar oxígeno.
 - Gestionar inventario.
 - Registrar progreso.
-

Item

Representa elementos que pueden ser utilizados o recolectados.

Ejemplos

- MedKit
 - OxygenTank
 - EnergyCell
-

Event

Representa situaciones que modifican el estado de la simulación.

Ejemplos

- Tormenta de radiación.
 - Ataque enemigo.
 - Hallazgo de recursos.
-

Simulation

Controla la ejecución principal de la aventura.

Responsabilidades

- Ejecutar ciclos de simulación.
 - Aplicar reglas.
 - Actualizar estados.
 - Determinar victoria o derrota.
-

Logger

Encargado de registrar los acontecimientos relevantes.

Responsabilidades

- Generar bitácora.
 - Registrar eventos.
 - Generar reportes finales.
-

5. Relaciones Entre Clases

La clase World contiene múltiples objetos Room.

Cada Room puede contener varios Item y Event.

Character interactúa con Room, Item y Event durante la simulación.

Simulation coordina todas las interacciones del sistema.

Logger registra las acciones generadas por Simulation.

6. Decisiones de Diseño

Se optó por una arquitectura modular basada en responsabilidades específicas.

Cada clase posee una función claramente definida para favorecer:

- Bajo acoplamiento.
- Alta cohesión.
- Facilidad de mantenimiento.
- Reutilización de código.

La lógica de simulación se encuentra separada de la representación de datos y de la generación de archivos de salida.

7. Manejo de Memoria y Recursos

El proyecto utiliza mecanismos modernos de C++ para administrar recursos de forma segura.

Se emplean punteros inteligentes cuando es necesario compartir objetos entre diferentes componentes del sistema.

Esto reduce el riesgo de:

- Fugas de memoria.
 - Liberaciones duplicadas.
 - Referencias inválidas.
-

8. Manejo de Archivos y Errores

El sistema valida la existencia y estructura de los archivos de entrada antes de iniciar la simulación.

Los errores detectados se manejan mediante excepciones y mensajes descriptivos para evitar fallos inesperados durante la ejecución.

Además, se generan archivos de salida para registrar la información producida durante la simulación.

9. Técnicas de Programación II Utilizadas

Durante el desarrollo se aplicaron diversos conceptos vistos en el curso:

- Programación Orientada a Objetos.
- Encapsulamiento.
- Abstracción.
- Relaciones entre clases.
- Sobrecarga de operadores.
- Manejo de archivos.
- Excepciones.
- Uso de STL.
- Manejo dinámico de memoria.

Estas técnicas permitieron construir una solución flexible y mantenible.

10. Evidencia de Ejecución

Durante la ejecución se generan:

- Bitácora de eventos.
- Reporte final.
- Salida en consola.

La información producida permite verificar el comportamiento completo de la simulación.

11. Limitaciones y Mejoras Futuras

Limitaciones

- Interfaz únicamente en consola.
- Cantidad limitada de eventos.
- IA simple para las decisiones automáticas.

Mejoras Futuras

- Mayor variedad de eventos.
 - Sistema de combate avanzado.
 - Generación procedural de mapas.
 - Interfaz gráfica.
 - Sistema de guardado y carga de partidas.
-

12. Conclusión

El proyecto cumple con los objetivos planteados al construir una simulación funcional basada en objetos, archivos de entrada y generación de resultados verificables. La arquitectura implementada permite futuras ampliaciones y demuestra la aplicación práctica de los conceptos estudiados en Programación II.